```python
import argparse
import sys
import os
import mlflow
import mlflow.sklearn
from sklearn.metrics import silhouette_score

from model_pipeline import (
    prepare_data,
    train_model,
    evaluate_model,
    save_model,
    load_model,
)


def main():
    parser = argparse.ArgumentParser(description="Delivery Time ML Pipeline")
    parser.add_argument("--data_path", type=str, required=True)
    parser.add_argument("--mode", choices=["train", "evaluate", "full"], default="full")
    parser.add_argument("--model_path", type=str, default="model_pipeline.joblib")

    args = parser.parse_args()

    if not os.path.exists(args.data_path):
        print("× Data file not found")
        sys.exit(1)

    # --- MLflow setup ---
    # Create a folder to store MLflow experiments/artifacts
    mlflow_dir = os.path.join(os.getcwd(), "mlruns")
    os.makedirs(mlflow_dir, exist_ok=True)

    # Set tracking URI to local folder
    mlflow.set_tracking_uri(f"file://{mlflow_dir}")

    # Set or create an experiment
    mlflow.set_experiment("KMeans_Experiment")

    # --- Training / Full mode ---
    if args.mode in ["train", "full"]:
        df, agg = prepare_data(args.data_path)

        # Example KMeans hyperparameters
        hyperparams = {
            "n_clusters": 3,
            "init": "k-means++",
            "max_iter": 300
        }

        with mlflow.start_run() as run:
            print("MLflow run started, run_id:", run.info.run_id)

            # Log hyperparameters
            for param_name, param_value in hyperparams.items():
                mlflow.log_param(param_name, param_value)

            # Train KMeans model
            model, scaler, agg, X_scaled = train_model(agg)

            # Evaluate clustering
            metrics = {
                "inertia": model.inertia_
            }

            # Optional: silhouette score
            if len(set(model.labels_)) > 1:
                metrics["silhouette_score"] = silhouette_score(X_scaled, model.labels_)

            # Log metrics
            for metric_name, metric_value in metrics.items():
                mlflow.log_metric(metric_name, metric_value)

            # Log model and scaler
            mlflow.sklearn.log_model(model, "kmeans_model")
            mlflow.sklearn.log_model(scaler, "scaler")

            print("✔ MLflow run finished")
            print("FINAL METRICS:", metrics)

        # Save model locally
        save_model(model, scaler, agg, args.model_path)

    # --- Evaluation mode ---
    elif args.mode == "evaluate":
        model, scaler, agg = load_model(args.model_path)
```

```
        print("Model loaded successfully")


if __name__ == "__main__":
    main()
```